



■ Descenso del Gradiente

- TLY1102

A black and white photograph of a man in a suit standing in a modern office, holding a tablet and smiling. The office has glass partitions and modern lighting. In the foreground, there is a desk with a coffee cup and some papers.

01

Introducción Losses

• Introducción

En términos matemáticos, **entrenar significa encontrar el theta (Θ) que minimiza la función de error (L).**

Minimizar la función de error es, encontrar los valores de los parámetros que entreguen el **menor valor para L .**

Usualmente, **minimizar una función se hace derivando e intentando igualar a cero.**

Lo que se hace es
tratar de acercarse al
menor error posible de
la función
 $L(\Theta)$

$$L = \frac{1}{n} \sum_{i=1}^n \text{error}(\hat{y}^{(i)}; y^{(i)})$$

$$\Theta = (w^{(1)}, b^{(1)}, w^{(2)}, b^{(2)}, \dots, U, C)$$

• Introducción

Funciones de pérdida para Regresión

MSE (Error Cuadrático Medio)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Pros: Penaliza severamente los errores grandes. Excelente cuando un error grande es inaceptable para tu problema.

Contras: Extremadamente sensible a valores atípicos (outliers). Un solo dato anómalo puede desviar significativamente el modelo.

MAE (Error Absoluto Medio)

$$MSE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Pros: Es robusta frente a los outliers. El error es más fácil de interpretar porque se mantiene en la misma unidad de medida que los datos originales.

Contras: A diferencia del MSE, no penaliza de forma extra los errores gigantes, tratándolos de forma lineal.

Regla de Oro: La escala de la variable importa. Si predices millones, la pérdida será de millones. Siempre debes escalar o normalizar los datos numéricos antes de entrenar.

• Introducción

Funciones de pérdida para Clasificación

Binary Crossentropy (Entropía Cruzada Binaria)

$$Loss = -\frac{1}{n} \sum_{i=1}^n y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)$$

Caso de Uso: Exclusivo para problemas de dos clases (ej. Fraude/No Fraude, 0 o 1).

Pros: Castiga exponencialmente las predicciones que están muy seguras de una clasificación incorrecta.

El Match Necesario: Requiere usar la función de activación Sigmoid en la capa de salida para generar probabilidades entre 0 y 1.

Categorical Crossentropy (Entropía Cruzada Categórica)

$$Loss = -\sum_{i=1}^c y_i \ln(\hat{y}_i)$$

Caso de Uso: Problemas multiclase (3 o más categorías, ej. clasificar tipos de animales).

Pros: Evalúa perfectamente distribuciones de probabilidad, asegurando que la clase correcta tenga la mayor confianza.

El Match Necesario: Requiere usar la función de activación Softmax en la capa de salida para que la suma de todas las probabilidades sea exactamente el 100%.

• Introducción

• Concepto de Derivada

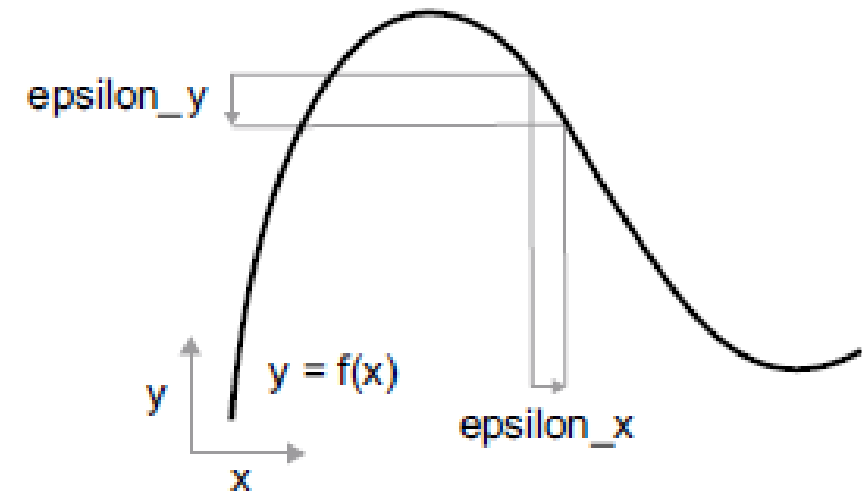
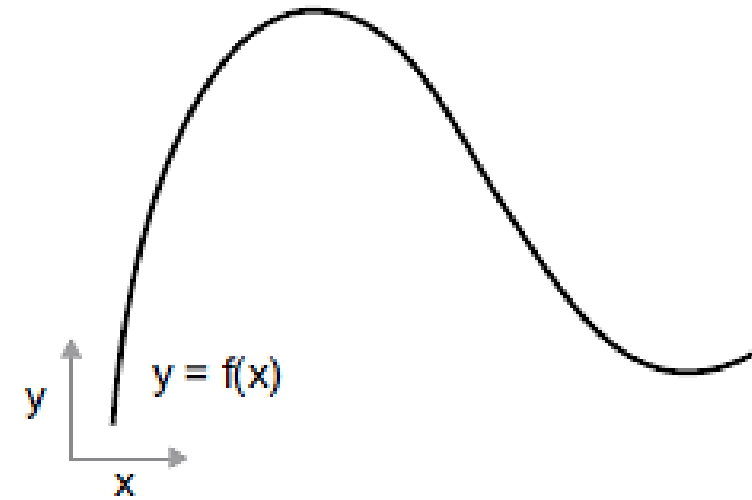
Consideremos una función continua y suave:

$$f(x) = y,$$

que asigna un número, x , a un nuevo número, y .

Como la función es continua, un pequeño cambio en x solo puede resultar en un pequeño cambio en y .

Digamos que aumentamos x por un factor pequeño, 'épsilon_x' esto resulta en un pequeño cambio de 'épsilon_y' en y .



• Introducción

Concepto de Derivada

Como la función es suave, cuando 'epsilon_x' es lo suficientemente pequeño, alrededor de un cierto punto p , es posible aproximar ' f ' como una función lineal de pendiente a , de modo que 'epsilon_y' se convierte en la fórmula que se indica. (Expansión de Taylor)

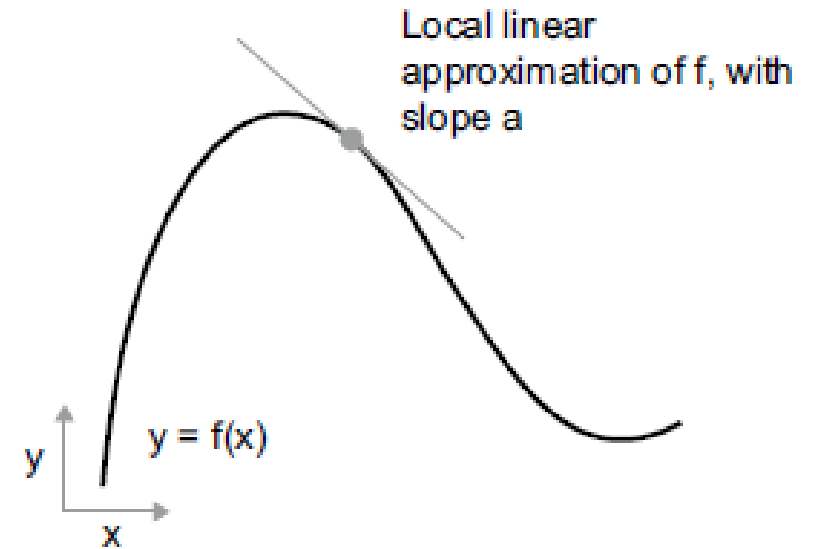
Obviamente, esta aproximación lineal es válida solo cuando x está lo suficientemente cerca de p .

La **pendiente ' a ' se denomina 'derivada de f en p '.**

Si a es negativa, significa que un pequeño aumento de x alrededor de p resultará en una disminución de $f(x)$, y si a es positiva, un pequeño aumento de x resultará en un aumento de $f(x)$.

Además, el valor absoluto de ' a ' (la magnitud de la derivada) indica la rapidez con la que se producirá este aumento o disminución.

$$f(x + \text{epsilon}_x) = y + a * \text{epsilon}_x$$



Fuente: Deep Learning with Python, F. Chollet



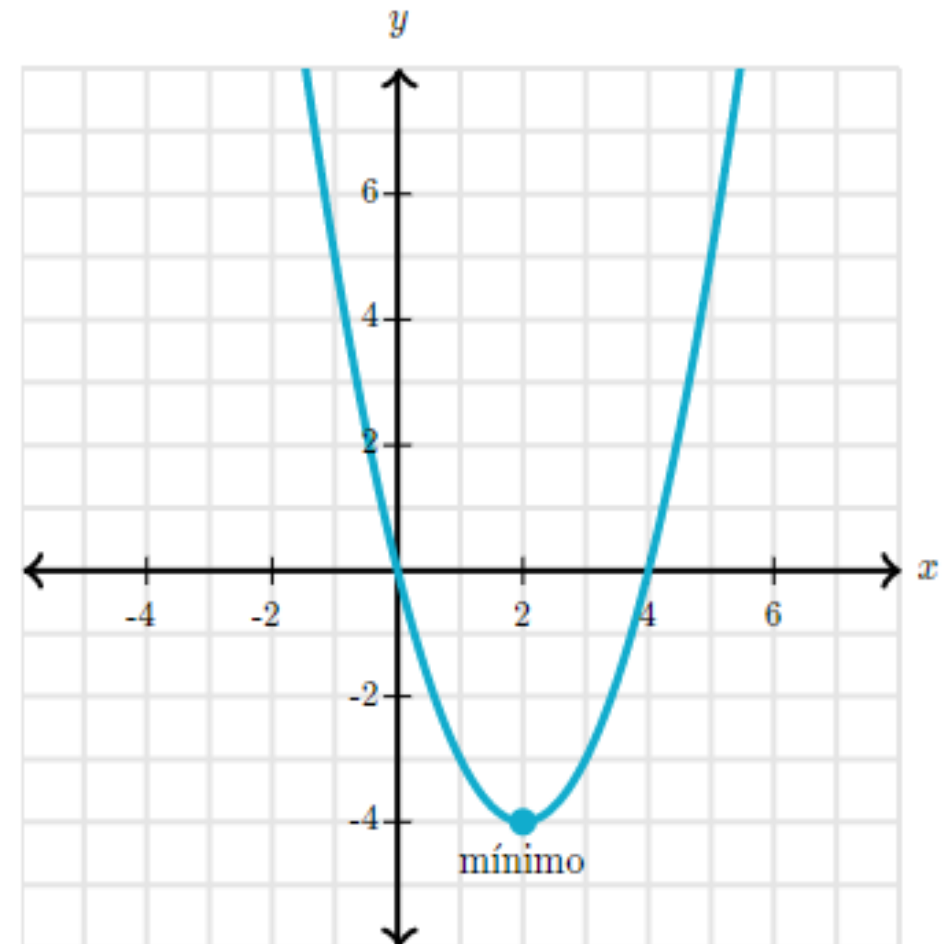
02

**Descenso del
gradiente**

El descenso de gradiente es un algoritmo que estima numéricamente dónde una función genera sus valores más bajos.

Eso significa que encuentra mínimos locales.

Para ello no encuentra mínimos manipulando símbolos, sino que el descenso de gradiente aproxima la solución con números.



Si tuviéramos una fórmula simple, entonces podríamos calcular fácilmente la derivada de la función igual a cero, determinando que $x = 4$, lo cual minimiza la función “ f ”.

Pero si nuestra función tiene cien o un millón de variables (como en un perceptrón o en una red feed forward), entonces manipular los símbolos no es factible.

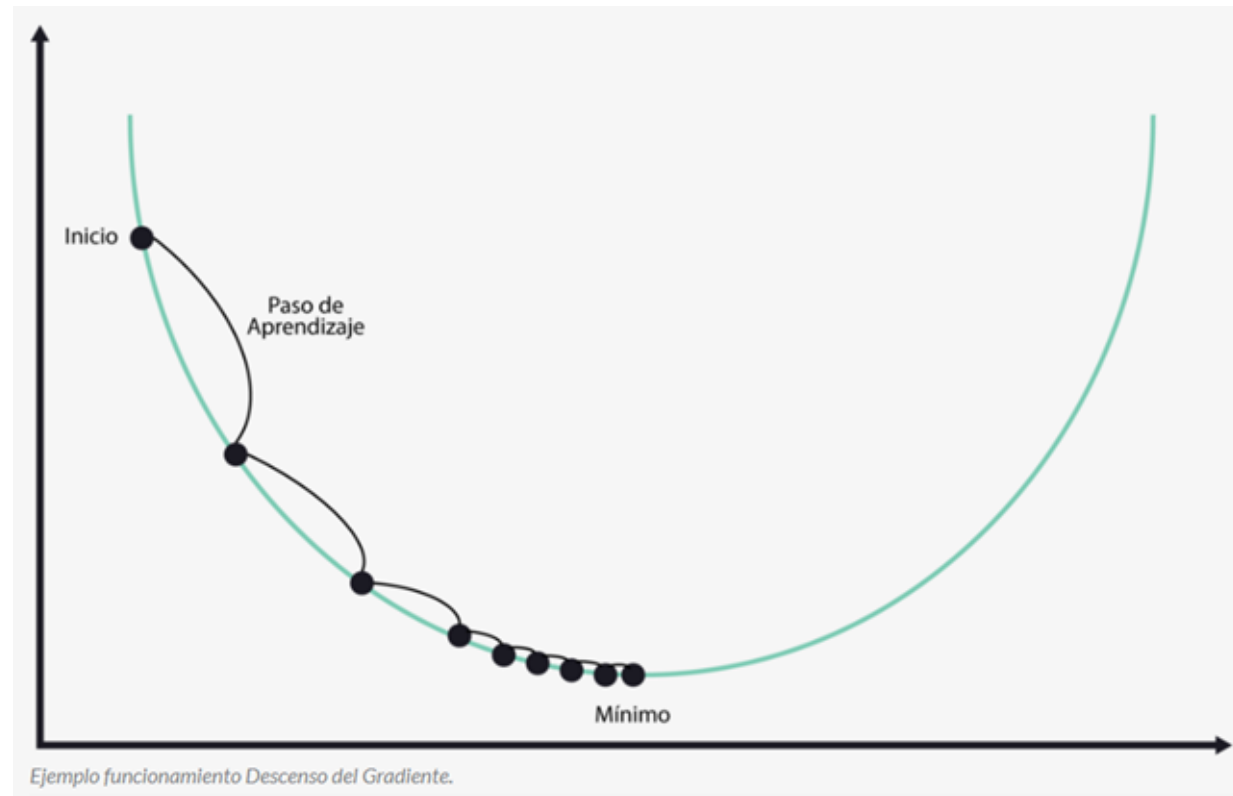
Ahí es cuando el descenso de gradiente puede darnos estas estimaciones sin importar cuán elaborada sea nuestra función.

$$f(x) = x^2 - 4x$$

$$\nabla f = 0$$

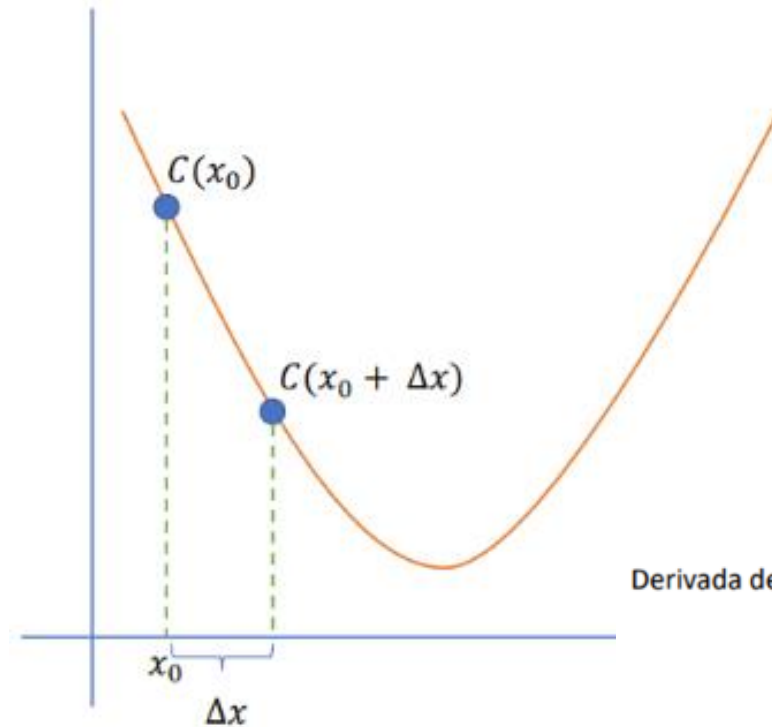
Imaginemos que estamos en una montaña en una noche nublada y sin ningún tipo de luz. Si quieres bajar de ahí lo más rápido posible, la mejor estrategia sería ver desde el punto donde estamos en qué dirección está la pendiente más inclinada y dar un paso hacia ahí. Repetiríamos este proceso hasta llegar abajo del todo.

El descenso de gradiente funciona igual. Esta técnica de optimización busca el mínimo de una función de esta manera, como se ve en la imagen.



Para bajar (o subir), se requiere calcular la derivada en cada punto de la curva.

En el caso de descenso del gradiente, queremos bajar para encontrar el punto mínimo de la curva.



Derivada de C en el punto x_0

$$\frac{\partial C}{\partial x} = \frac{C(x_0 + \Delta x) - C(x_0)}{\Delta x} = \frac{\Delta C}{\Delta x}$$



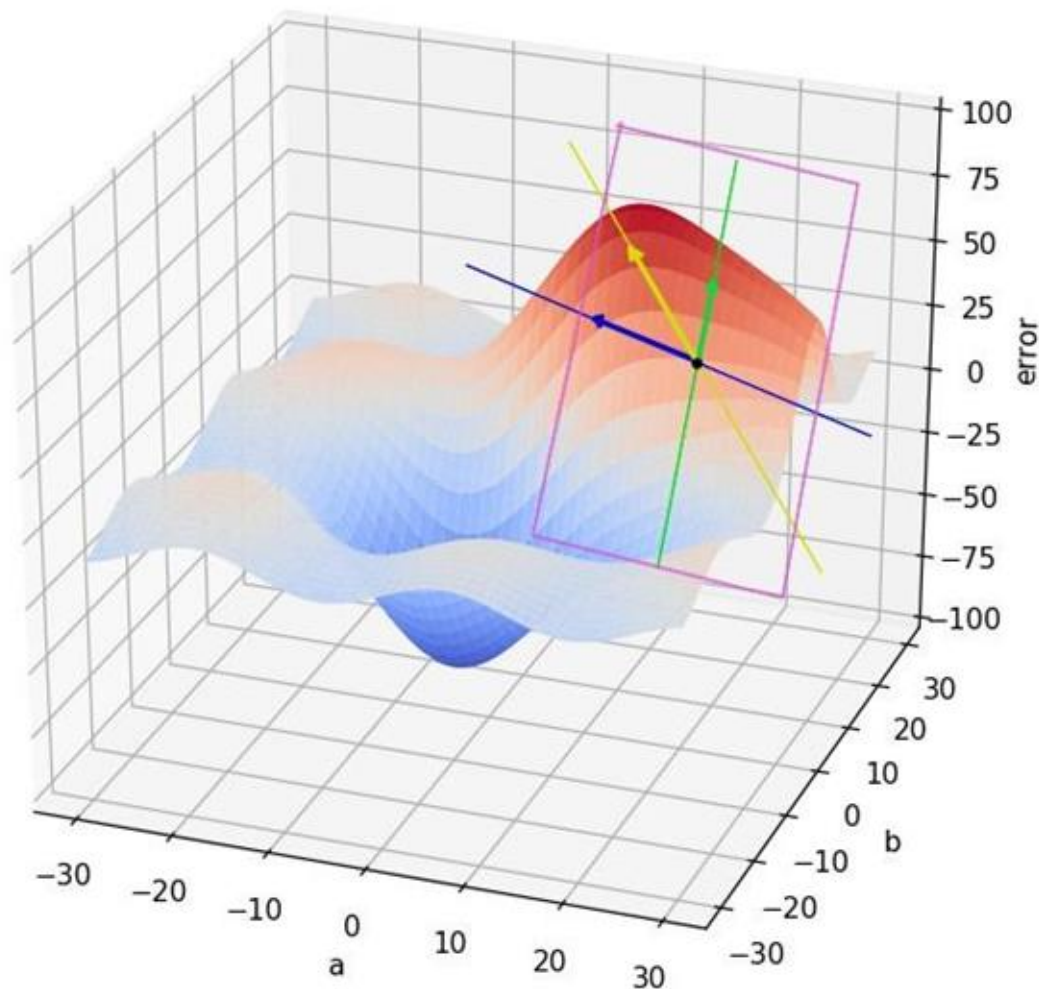
$$\Delta C = \frac{\partial C}{\partial x} \Delta x$$

El cambio en la función depende de la derivada de la función y el cambio en los parámetros.

El gradiente evaluado en cualquier punto representa la dirección del ascenso más pronunciado por este terreno montañoso.

Para minimizar la función, podemos seguir el negativo del gradiente, y así ir en la dirección del descenso más pronunciado. Este es el descenso de gradiente.

Por el contrario, si nos vamos por el lado positivo del gradiente, estamos maximizando la función





03

Funcionamiento

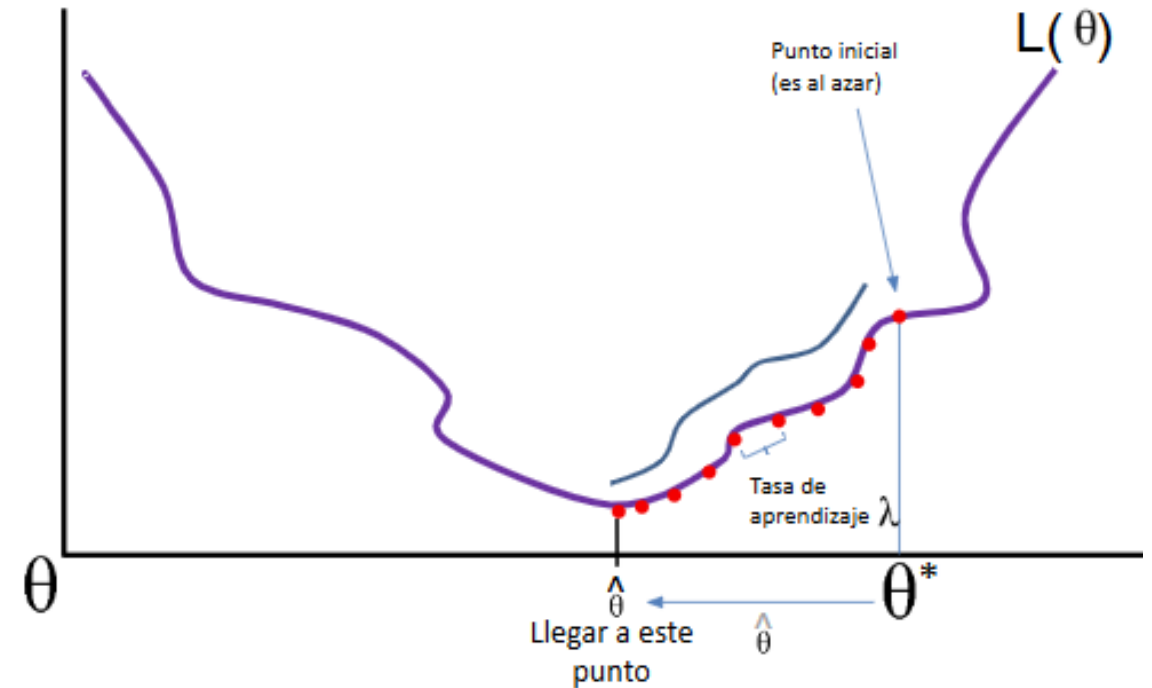
Funcionamiento

El algoritmo **comienza en un punto al azar** (el valor inicial de $W_{i,j}$) dentro de la función.

El objetivo **es iterar**, hasta llegar al punto $\hat{\theta}$, que **es el punto que minimiza la función**.

En el paso a paso, cada vez que ajusto o cambio el $W_{i,j}$, **se calcula la derivada en este punto**.

(Derivar significa encontrar la pendiente de la función en ese punto).



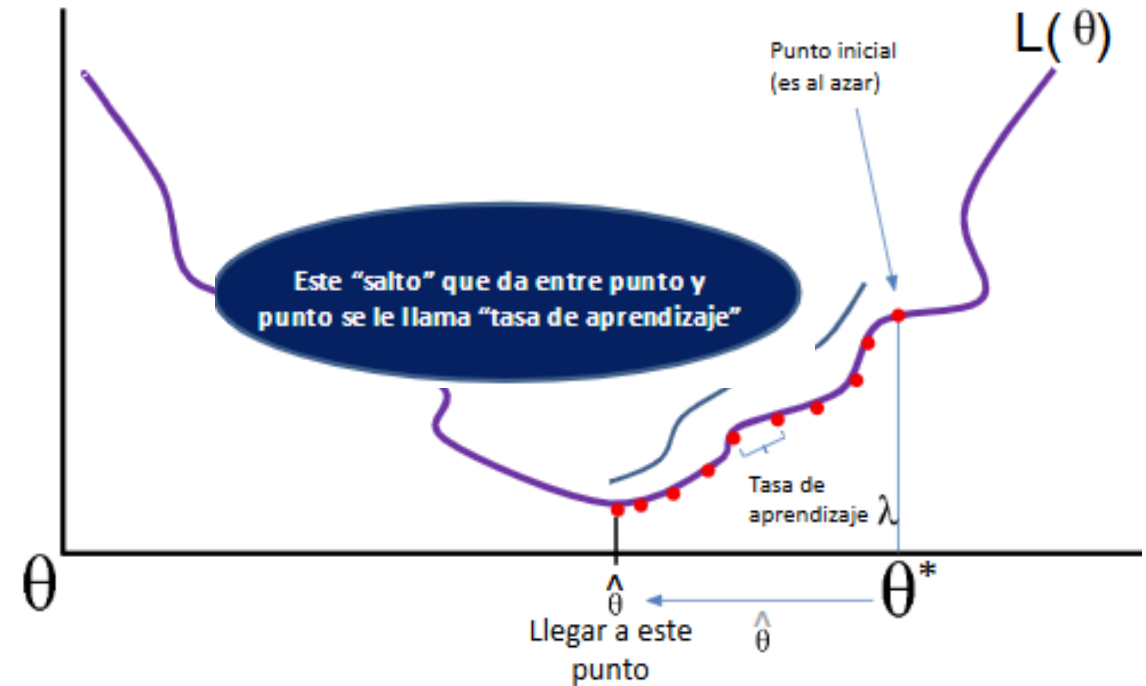
Funcionamiento

TASA DE APRENDIZAJE

Al cambiar el valor del parámetro, desde ese nuevo punto se calcula la derivada a cero y luego se compara en qué sentido la función L crece y en cuál decrece.

La idea es **mover la función en la dirección contraria a la que crece la pendiente** (es decir en sentido decreciente).

Cada “salto” o espacio que la función se mueve es denominada “**tasa de aprendizaje**”, que indica en cuánto debo mover el parámetro.

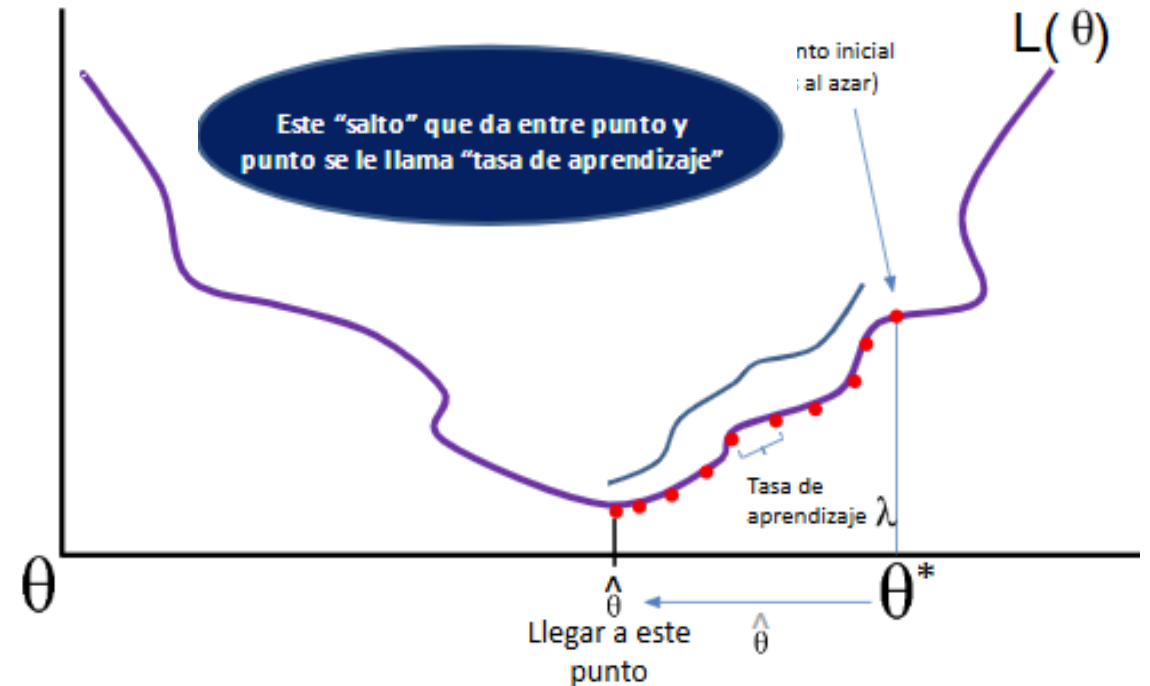


Funcionamiento

Luego, se vuelve a cambiar el valor del parámetro y se itera tantas veces como se necesite **hasta que el resultado sea igual a cero**, o en la “vida real”, hasta cuando alcance el criterio que se ha definido para detener el algoritmo.

El trabajo que realmente es complicado en el descenso del gradiente es calcular las derivadas, es lo realmente caro computacionalmente.

En una red con miles o millones de parámetros (pesos y bias), calcular el error “es muy caro” ya que se debe pasar todos los ejemplos.



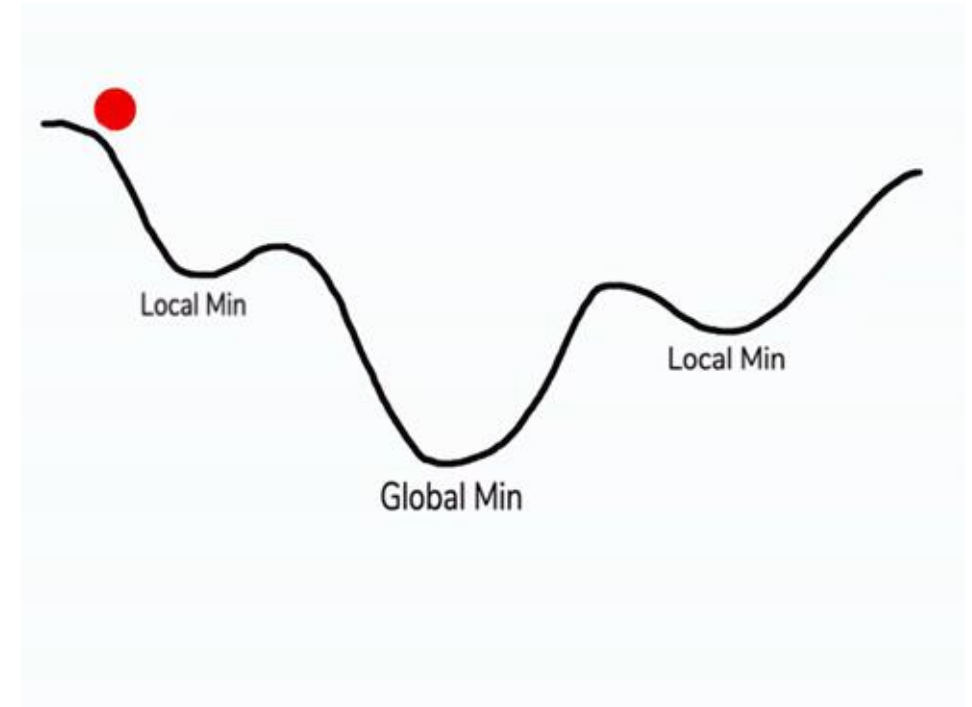
Funcionamiento

MINIMO LOCAL

Un problema del descenso del gradiente es el “Mínimo Local”, es decir, que el menor valor que el algoritmo encuentre en un punto sea o no el mínimo de la función (mínimo global).

Esto en la práctica, no tiene solución.

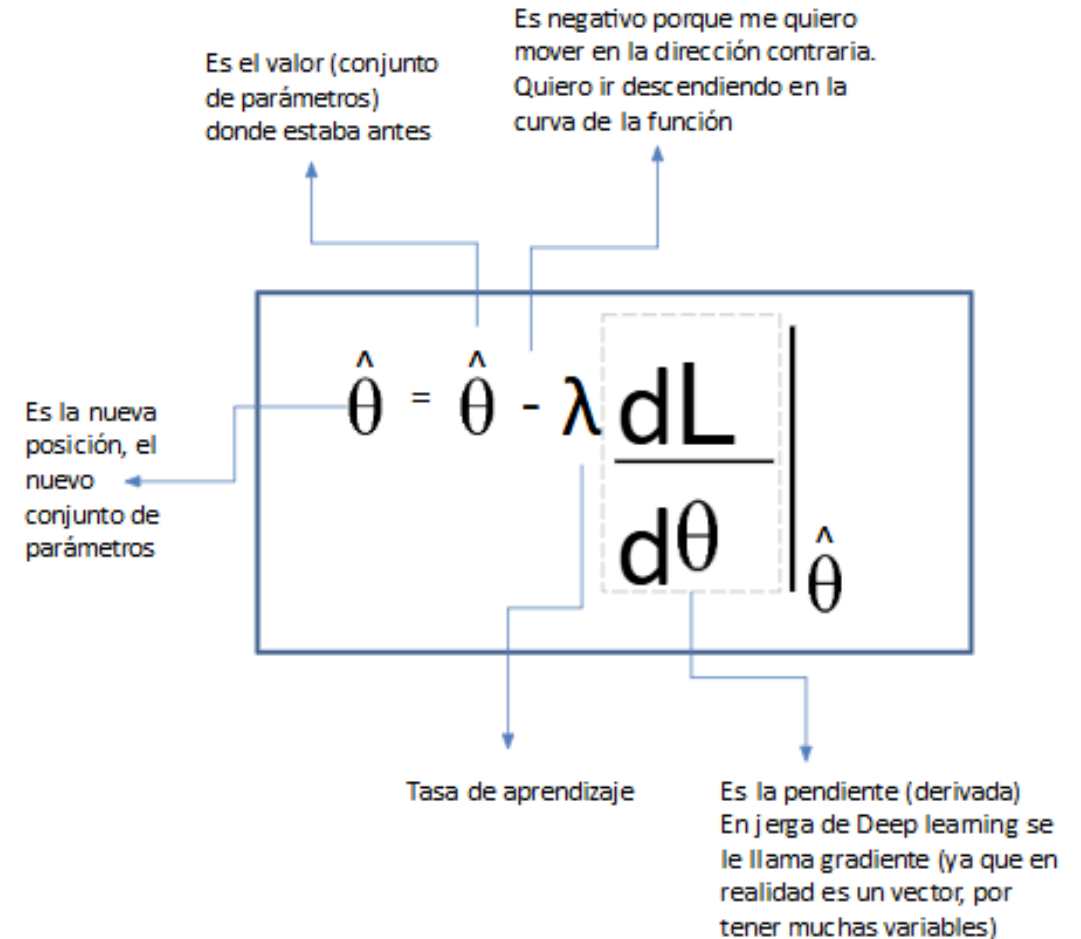
No hay forma de saber si el mínimo encontrado es un mínimo local o un mínimo global.



Funcionamiento

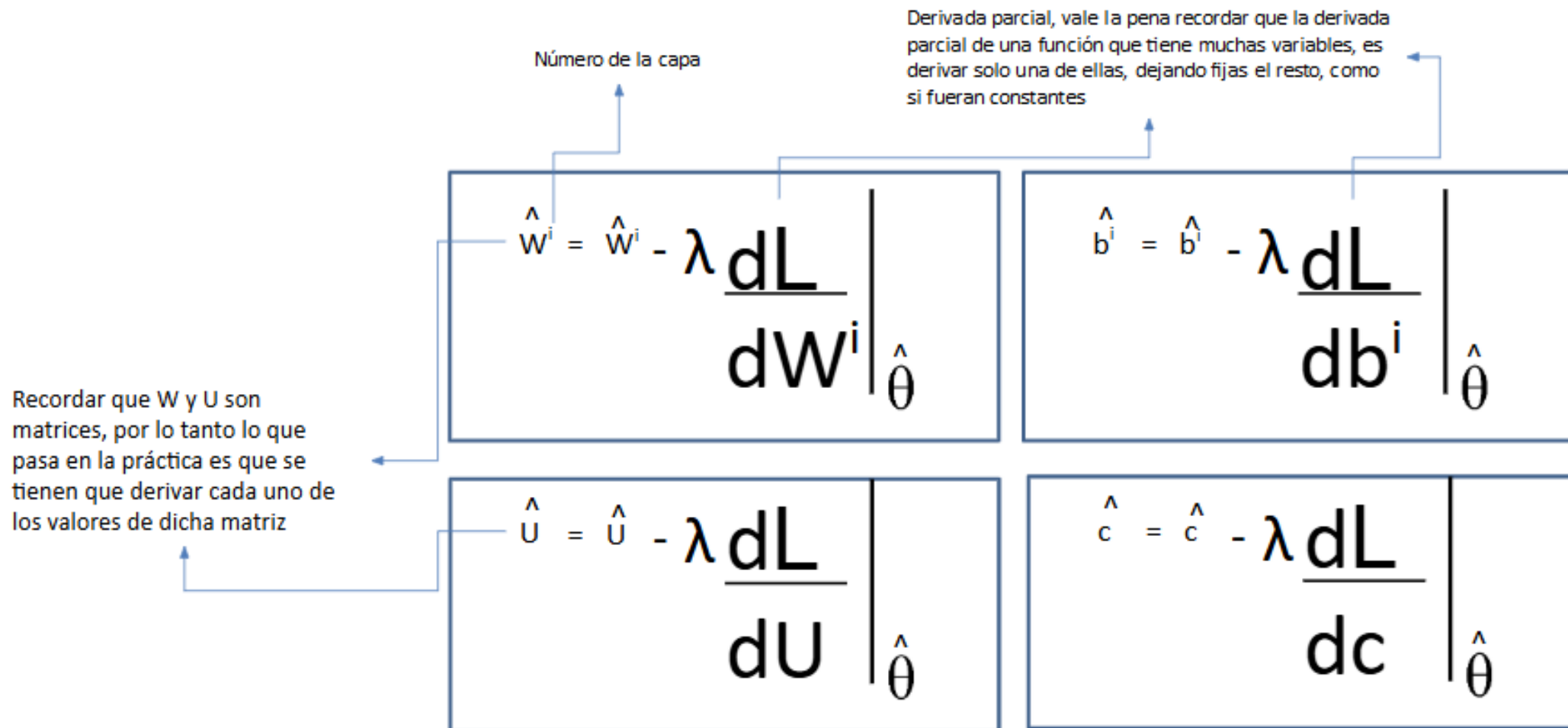
Matemáticamente, esta sería la fórmula, donde se indica que, dado un conjunto Θ de parámetros, se evalúa la pendiente de la función y se invierte el sentido (me muevo en la dirección contraria), dando saltos de largo λ y el proceso termina al alcanzar el criterio definido (la función es minimizada).

En otras palabras, se actualiza el Θ (conjunto de parámetros) con el Θ de la iteración anterior y se calcula la derivada del error con respecto a Θ , evaluada en $\hat{\Theta}$, saltando en la dirección contraria dando un salto definido por λ .



Funcionamiento

Generalizando la fórmula del descenso del gradiente a la estructura general de la red:



04

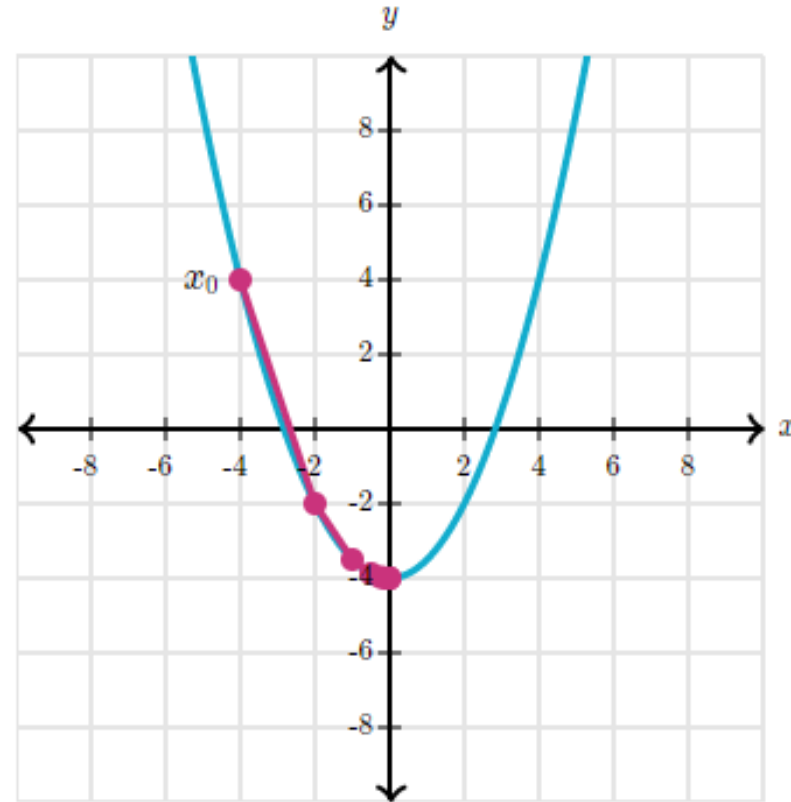
Consideraciones



• Tasa de Aprendizaje

Una limitación del descenso de gradiente tiene que ver con el tamaño de paso α .

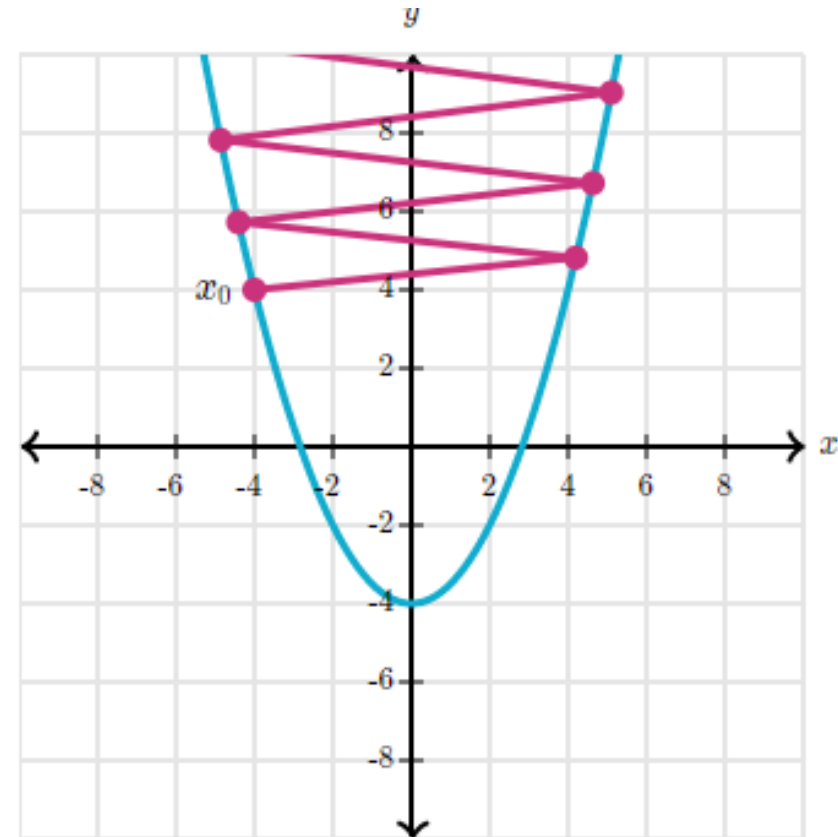
Un buen tamaño de paso se mueve hacia el mínimo rápidamente y con cada paso realiza un progreso sustancial.



Un buen tamaño del paso converge rápidamente.

• Tasa de Aprendizaje

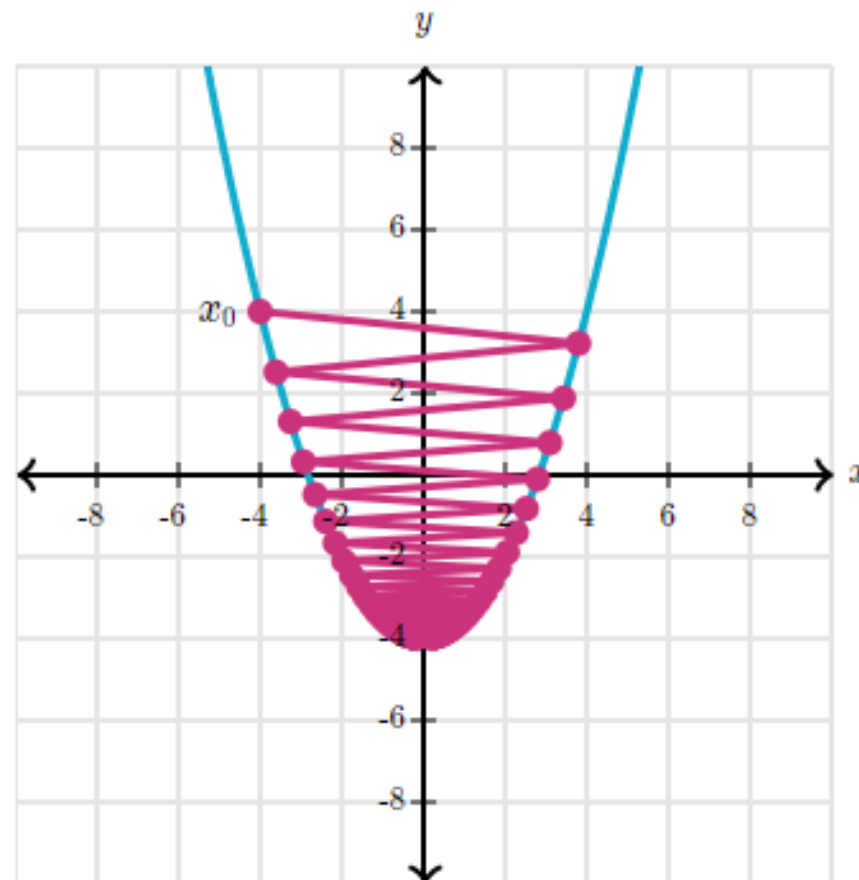
Sin embargo, si el tamaño de paso es demasiado grande, es posible que nunca converjamos al mínimo local porque lo excedemos en cada ocasión.



Un tamaño de paso grande diverge.

• Tasa de Aprendizaje

Si tenemos suerte y el algoritmo converge de todos modos, aun así, podría dar más pasos de los necesarios.

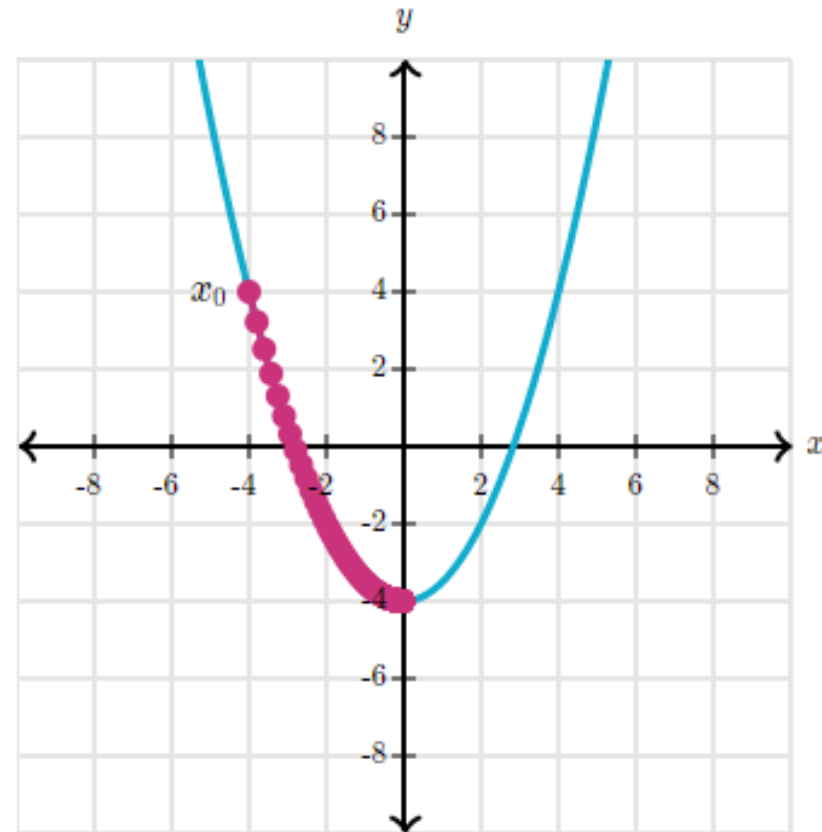


Un tamaño de paso grande converge lentamente.

• Tasa de Aprendizaje

Si el tamaño de paso es demasiado **pequeño**, entonces tendremos una mayor probabilidad de que converja, pero tomaremos muchos más pasos de los necesarios.

Esto es un problema cuando la función que estamos minimizando tiene miles o millones de variables, y evaluarla es engorroso.



Un tamaño de paso muy pequeño converge lentamente.

• Descenso Estocástico

Aunque es posible resolver ecuaciones de hasta tres dimensiones con el descenso del gradiente, hacerlo para redes neuronales reales, donde el número de parámetros nunca es inferior a unos pocos miles y puede a menudo ser de varias decenas de millones, es imposible.

De allí, debemos utilizar un algoritmo de aproximación denominado **descenso del gradiente estocástico 'mini-batch'**.

El término estocástico se refiere al hecho de usar lote de datos al azar (estocástico es un sinónimo científico de aleatorio).

• Descenso Estocástico

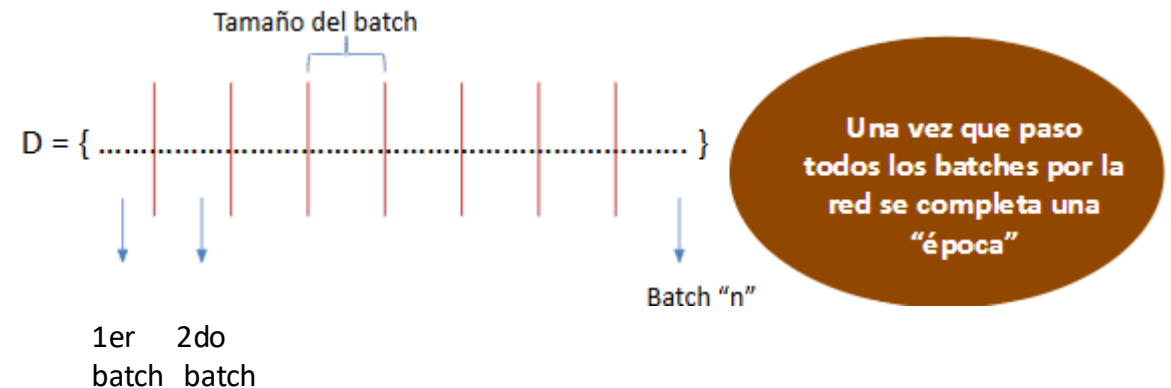
En lugar de calcular el error promedio de todos los datos, lo que se hace es “estimar” el error de grupos de datos denominados lotes o batch.

Para hacer esto, se toma un batch de los ejemplos y estos se usan para estimar el error.

En otras palabras, no se espera a que pasen todos los datos para actualizar los parámetros, sino que se hace sobre una porción de ellos.

Es importantísimo que este batch sea al “azar”.

- En la práctica, se toman todos los ejemplos, se “desordenan”, se ponen en una lista, se define el tamaño del batch... y dividir el set de datos en el tamaño de este batch
- Se toma el primer batch, se pasa por la red, se calcula el error y se ajecutan los parámetros y luego...
- Se toma el segundo batch, se pasa paso por la red, se calcula el error y se ajustan los parámetros
- Y así sucesivamente hasta pasar todos los batches



DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO



NIVEL DE EXCELENCIA

HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la calidad / Vinculación con el Medio / Investigación, creación y/o innovación